LINUX INTERVIEW PREPARATION GUIDE

CHEATSHEET

Q&A

Linux Interview Quick Revision

Last-minute revision for DevOps, SysAdmin and Cloud Engineer interviews (1–5 years of experience)

IN THIS CHEATSHEET

01 Linux Fundamentals
02 Processes and Signals
03 Permissions
04 Storage
05 Networking
06 Services and Logging
07 Bash Scripting
08 Docker and Kubernetes
09 AWS for Linux Engineers
10 Essential Commands
11 Troubleshooting Scenarios
12 Top Interview Traps
13 Checklist and Takeaways

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHEATSHEET OBJECTIVE

Revise the answers interviewers expect, in the order you would give them. Every section gives a short answer first, then the trap or follow-up question that usually comes next. Each section also points to the chapter and lab where the topic is covered in depth.

1 Linux Fundamentals

Deep dive: Chapters 1, 8 and 14 · Practice: Lab 1

Question	Quick answer
What is Linux?	An open-source, Unix-like kernel. A **distribution** (Ubuntu, RHEL, Amazon Linux) adds user-space tools, a package manager and an init system to make a full OS.
What is the kernel?	The core that manages CPU scheduling, memory, processes, devices, filesystems and networking.
User space vs kernel space?	Applications run in unprivileged user space. The kernel runs in privileged kernel space. Programs ask the kernel for services through **system calls** (`open` , `read` , `fork`).
Shell vs terminal?	The terminal is the window that shows text. The shell (Bash, zsh) interprets commands and runs programs.
What is Bash?	The most common Linux shell, and also a scripting language.
What is a PID?	The unique number the kernel gives each running process.
What is PID 1?	The first user-space process. It starts services and adopts orphaned processes. On servers it is usually systemd; in a container it is the container's main process.
What is root?	The superuser. The kernel checks **UID 0**, not the name "root".
What is a daemon?	A background process that provides a service, such as `sshd` or `nginx` .
Boot order?	BIOS/UEFI → GRUB → kernel + initramfs → PID 1 (systemd) → targets and services → login.

INTERVIEW TRAP

"Why does my container take 10 seconds to stop?" The app runs as PID 1, and the kernel gives PID 1 no default signal handlers. If the app does not handle SIGTERM, it ignores it, and Docker sends SIGKILL after the timeout. Fix it by handling SIGTERM in the app, using `exec` in the entrypoint script, or running a small init with `docker run --init`.

2 Processes and Signals

Deep dive: Chapters 5 and 13 · Practice: Labs 3 and 8

State	Meaning	What it tells you
`R`	Running or runnable	Using a CPU or waiting for one
`S`	Interruptible sleep	Normal idle waiting
`D`	Uninterruptible sleep	Usually blocked on I/O; counts towards load average
`T`	Stopped	Paused by SIGSTOP or Ctrl+Z
`Z`	Zombie	Exited, but the parent has not collected its exit status

Signal	Number	Use
SIGHUP	1	Terminal closed; many daemons treat it as "reload config"
SIGINT	2	Ctrl+C
SIGKILL	9	Immediate kill; cannot be caught, blocked or ignored
SIGTERM	15	Polite "please exit"; the default for `kill`
SIGSTOP	19 (x86)	Pause; cannot be caught. Numbers vary by CPU architecture, so prefer names

Q. Process vs thread?

A process is a running program with its own address space. Threads run inside a process and share its memory and open files, which makes them cheaper to create but means one bad thread can crash the whole process.

Q. Can you kill a zombie process?

No. It is already dead; only its process-table entry remains. The parent must reap it with `wait()`. Fix the parent, or kill the parent so PID 1 adopts and reaps the zombie. Find zombies and their parents with `ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/'`.

Q. Why did `kill -9` not stop a process?

It is either a zombie, which is already dead, or in D state, which is waiting in the kernel for I/O such as a hung NFS mount or failing disk. The signal is delivered only when the I/O completes. The fix is the storage problem, not more signals.

Q. SIGTERM vs SIGKILL?

SIGTERM lets the process finish requests, flush data and remove lock files. SIGKILL stops it instantly with no cleanup. Always send SIGTERM first and use SIGKILL only if the process does not exit within a reasonable time.

3 Permissions

Deep dive: Chapters 3 and 9 · Practice: Lab 2

Bit	On a file	On a directory
`r` (4)	Read the contents	List the names inside
`w` (2)	Modify the contents	Create, delete and rename entries (needs `x` too)
`x` (1)	Execute	Enter (`cd`) and reach files inside

Mode	Symbolic	Typical use
`755`	`rwxr-xr-x`	Scripts, binaries, web directories
`644`	`rw-r--r--`	Config files, web content
`600`	`rw-------`	Private keys, `authorized_keys` , secrets
`700`	`rwx------`	`~/.ssh`
`2770`	`rwxrws---`	Team directory: setgid keeps the group
`1777`	`rwxrwxrwt`	`/tmp` : sticky bit

Q. What is umask?

A mask of bits **removed** from new files and directories. Files start from 666 and directories from 777, so umask `022` gives 644 and 755. It is a bitwise mask, not subtraction: umask `033` on a new file gives 644, not 633.

Q. SUID, SGID and sticky bit?

SUID runs a binary as its owner (`/usr/bin/passwd` runs as root to update `/etc/shadow`). **SGID** on a directory makes new files inherit the directory's group. The **sticky bit** on a directory lets only a file's owner delete it (`/tmp`). Linux ignores SUID on scripts.

Q. What are ACLs?

Extra per-user and per-group permissions beyond owner/group/other, managed with `getfacl` and `setfacl` . A `+` at the end of `ls -l` permissions means an ACL exists. Default ACLs on a directory apply to new files created inside it.

4 Storage

Deep dive: Chapter 6 · Practice: Lab 4

```
Disk / EBS volume → Partition (optional) → LVM: PV → VG → LV → Filesystem (ext4 / XFS) → M
fstab line:  UUID=...   /data   ext4   defaults,nofail   0   2
             device     mount   type   options           dump fsck-order
```

Question	Quick answer
`df` vs `du`	`df` asks the filesystem how many blocks are used. `du` walks the directory tree and adds up file sizes. They differ when files are deleted but still open, or hidden under a mount point.
Inode	Stores a file's metadata: owner, mode, timestamps, size and block pointers. It does **not** store the filename; the name lives in a directory entry.
Hard vs soft link	A hard link is another name for the same inode (same filesystem, no directories). A symlink is a small file that stores a path and breaks if the target is removed.
LVM	Pools disks into volume groups so you can grow logical volumes online: `lvextend -r -L +10G /dev/vg/lv`.
ext4 vs XFS	ext4 (Ubuntu default) grows with `resize2fs` and can shrink offline. XFS (RHEL default) grows with `xfs_growfs` and **cannot shrink**.
Why UUID and `nofail` in fstab?	Device names can change between boots. `nofail` stops a missing data disk from dropping the server into emergency mode.

```
df -h          # space per filesystem
df -i          # inodes: "No space left" with free space shown = inode exhaustion
du -sh /var/* 2>/dev/null | sort -h | tail
sudo lsof +L1  # deleted files still held open
lsblk -f       # disks, filesystems, UUIDs, mount points
findmnt --verify
```

5 Networking

Deep dive: Chapter 7 · Practice: Lab 5

Topic	TCP	UDP
Connection	Three-way handshake (SYN, SYN-ACK, ACK)	Connectionless
Delivery	Reliable, ordered, retransmits	Best effort, no ordering
Used by	SSH, HTTP/HTTPS, databases	DNS queries, NTP, syslog, streaming

Question	Quick answer
DNS	Translates names to IPs and holds other records (CNAME, MX, TXT). It uses UDP 53, plus TCP 53 for large answers and zone transfers.
Default gateway	The router used for any destination not on a local subnet (`ip route` → `default via`).
Refused vs timed out	**Refused:** the host answered, but nothing is listening or a REJECT rule fired. **Timed out:** packets were silently dropped by a firewall, security group, NACL or bad route.
`0.0.0.0` vs `127.0.0.1`	Listening on all interfaces vs this machine only. A service bound to 127.0.0.1 can never be reached remotely.
Common ports	22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 123 NTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB. These are defaults; applications can be configured to use others.

```
sudo ss -tulnp                  # listening sockets and owning processes
nc -zv -w 3 SERVER_IP 443       # test a remote TCP port
curl -v http://localhost        # full HTTP conversation
dig example.com +short
getent hosts example.com        # what applications actually resolve
ip -br addr; ip route
sudo tcpdump -i any port 443 -c 20
```

6 Services and Logging

Deep dive: Chapters 5 and 12 · Practice: Labs 3, 6 and 7

Command	What it does
`systemctl start / stop`	Change the state now
`systemctl restart`	Stop then start: new process, dropped connections
`systemctl reload`	Re-read config without stopping, if the service supports it
`systemctl enable / disable`	Start or don't start at boot; does **not** change the current state
`systemctl enable --now`	Enable at boot and start now
`systemctl mask`	Block the unit from being started at all
`systemctl daemon-reload`	Re-read unit files after editing them
`systemctl --failed`	List failed units

```
journalctl -u nginx -n 100 --no-pager     # last 100 lines of one service
journalctl -u nginx -f                    # follow live
journalctl -p err -b                      # errors since the last boot
journalctl -k                             # kernel messages (OOM kills, disk errors)
journalctl --since "1 hour ago"
```

Item	Ubuntu	RHEL-family
System log	`/var/log/syslog`	`/var/log/messages`
Auth log	`/var/log/auth.log`	`/var/log/secure`
Cron service	`cron`	`crond` (not installed by default on Amazon Linux 2023)

Feature	cron	systemd timer
Setup	One crontab line	A `.service` file plus a `.timer` file
Logging	Only what you redirect	Automatic in the journal
Missed runs (server was off)	Skipped	Run at boot with `Persistent=true`
Overlap, dependencies, limits	Manual (`flock`)	Built in (unit dependencies, resource limits)

```
*  *  *  *  *   command          Examples:   0 2 * * *     daily at 02:00
|  |  |  |  +-- day of week (0-7, Sun = 0 or 7)        */5 * * * *   every 5 minutes
|  |  |  +----- month (1-12)                           0 3 * * 0     Sundays at 03:00
|  |  +-------- day of month (1-31)
|  +----------- hour (0-23)
+-------------- minute (0-59)
```

7 Bash Scripting

Deep dive: Chapters 10 and 11 · Practice: Lab 6

Variable	Meaning
`$0`	Script name
`$1`, `$2` ...	Positional arguments
`$#`	Number of arguments
`"$@"`	All arguments, each kept as a separate word (use this one)
`"$*"`	All arguments joined into one string
`$?`	Exit status of the last command (0 = success)
`$$` / `$!`	PID of this shell / PID of the last background job

Exit code	Meaning
`0`	Success
`1`, `2`	General error, misuse of arguments
`126` / `127`	Not executable / command not found
`128 + n`	Killed by signal n (137 = SIGKILL, 143 = SIGTERM)

```
#!/bin/bash                  # shebang: which interpreter runs the script
set -Eeuo pipefail           # -e exit on error, -u unset vars are errors,
                             # -o pipefail a failing pipe stage fails the pipe, -E keep E
trap 'echo "Failed at line $LINENO" >&2' ERR
```

INTERVIEW TRAP

`set -e` does **not** exit for a failing command inside an `if` condition, a `while` test, or a `cmd1 && cmd2` list. It is a safety net, not error handling. Check the important commands explicitly, and turn `-e` off in scripts like health checks that must keep going after one check fails.

8 Docker and Kubernetes

Deep dive: Chapters 14 and 15 · Practice: Lab 9

Question	Quick answer
Container vs VM	A container is an isolated process that **shares the host kernel**. A VM runs its own kernel on a hypervisor.
Namespaces vs cgroups	Namespaces control what a process can **see** (PID, network, mount, user). cgroups control how much it can **use** (CPU, memory, I/O).
Image vs container	An image is a read-only, layered template. A container is a running instance with a thin writable layer on top (OverlayFS).
Docker volume	Docker-managed storage under `/var/lib/docker/volumes` that outlives the container. A bind mount maps a host path instead.
Pod	The smallest Kubernetes unit: one or more containers sharing a network namespace and volumes.
Deployment	Keeps the desired number of identical Pod replicas running and handles rolling updates.
Service	A stable virtual IP and DNS name in front of a changing set of Pods.
CrashLoopBackOff	The container keeps exiting; Kubernetes waits longer before each restart. Check `kubectl logs --previous`.
ImagePullBackOff	The image cannot be pulled: wrong tag, missing registry credentials or no network egress.
OOMKilled	The container went over its memory limit and the kernel killed it with SIGKILL, so the exit code is **137**.
CPU limit reached?	The container is **throttled** (runs slower), not killed. Only memory is enforced by killing.

9 AWS for Linux Engineers

Deep dive: Chapter 17 · Practice: Lab 10

Service	Purpose	Linux angle interviewers ask about
EC2	Virtual servers	User data runs once as root at first boot; IMDSv2 needs a session token
EBS	Block storage for one instance (same AZ)	Appears as `/dev/nvme1n1` on Nitro; grow with `growpart` + `resize2fs` / `xfs_growfs`
EFS	Shared NFS file storage across instances and AZs	Mounted over NFS (port 2049); a hung mount leaves processes in D state
S3	Object storage	`aws s3 cp` with an instance role, never hard-coded keys
IAM	Identity and permissions	Instance profile; `aws sts get-caller-identity` shows who you really are
VPC	Isolated virtual network	Public subnet = route to an internet gateway; private subnet uses NAT or endpoints
Security group	Stateful firewall on the instance's network interface	Return traffic is allowed automatically; allow rules only
NACL	Stateless filter at the subnet level	Must allow return traffic on ephemeral ports (1024–65535)
CloudWatch	Metrics, logs and alarms	Memory and disk usage are **not** collected by default; install the CloudWatch agent
Systems Manager	Fleet management and Session Manager	Shell access with no inbound port 22: needs the agent, an IAM role and outbound 443

10 Essential Commands

Deep dive: Chapters 2, 5, 11 and 13

```
# System
cat /etc/os-release; uname -r; hostnamectl; uptime
lscpu; nproc; free -h

# Disk
df -h; df -i; lsblk -f; findmnt

# Processes
ps aux --sort=-%cpu | head; ps aux --sort=-%mem | head
top; pgrep -a nginx; pidof nginx

# Services and logs
systemctl status nginx; systemctl --failed
journalctl -u nginx -n 100 --no-pager; journalctl -k

# Network
ip -br addr; ip route; sudo ss -tulnp
dig example.com +short; getent hosts example.com
curl -I http://localhost; nc -zv -w 3 SERVER_IP 22

# Performance
vmstat 1 5; iostat -xz 1 5; sar -u 1 5

# Files and permissions
ls -l; namei -l /path/to/file; getfacl /path; sudo lsof +L1

# Remote access
ssh -v user@SERVER_IP
```

Item	Ubuntu / Debian	RHEL / Rocky / Amazon Linux 2023
Package manager	`apt` , `dpkg -l`	`dnf` , `rpm -qa`
Which package owns a file	`dpkg -S /path`	`rpm -qf /path`
SSH service	`ssh`	`sshd`
Host firewall	UFW	firewalld (none by default on Amazon Linux 2023)
Default root filesystem	ext4	XFS
Admin group	`sudo`	`wheel`
Mandatory access control	AppArmor	SELinux (`getenforce`)

11 Production Troubleshooting Scenarios

Deep dive: Chapter 16 · Practice: Labs 3–12 and the Capstone

Scenario	First commands	What you are looking for
Website down	`curl -I`, `systemctl status`, `ss -tlnp`, `error.log`	Refused vs 5xx; service state; port and bind address
Nginx 502	`tail error.log`, upstream service status	Upstream down, wrong port, PHP socket, SELinux boolean
Disk full	`df -h`, `df -i`, `du`, `lsof +L1`	Large logs, inode exhaustion, deleted-but-open files
High CPU	`uptime`, `top`, `ps --sort=-%cpu`	Load vs cores; `us` vs `sy` vs `wa` vs `st`
High memory	`free -h`, `ps --sort=-%mem`, `vmstat`	`available`, swap in/out, OOM kills in `journalctl -k`
Application stopped	`systemctl status`, `journalctl -u`	Exit code, crash message, restart loop
Port conflict	`ss -tlnp 'sport = :80'`, `lsof -i :80`	Which process already owns the port
SSH failure	`ssh -v`, `nc -zv`, auth log	Timeout (network) vs refused (sshd) vs publickey (key, user, permissions)
DNS failure	`getent hosts`, `dig`, `resolvectl status`	Resolver config, `/etc/hosts` overrides, wrong record
Permission denied	`id`, `ls -l`, `namei -l`, `getfacl`	The path component that blocks access; SELinux (`ls -Z`)
TLS/SSL error	`curl -Iv https://host`, `openssl s_client`	Expiry date, hostname mismatch, missing intermediate certificate

```
# Certificate expiry and chain in one line
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null
  | openssl x509 -noout -subject -issuer -dates
```

How to answer any troubleshooting question

```
1. Clarify   → What exactly fails, for whom, since when? What changed recently?
2. Scope     → One user or all? One server or all? Is the issue local or remote?
3. Layer     → Network → host resources → service → application → dependencies
4. Evidence  → Name the command, what you expect to see, and what it would prove
5. Fix safely→ Smallest reversible change; validate config before reload; roll back if ne
6. Prevent   → Alert, runbook, root cause write-up
```

Scenario 1 **"The website is down. What do you do?"**

Model answer: First I reproduce the problem with `curl -I` from the server and from outside, which tells me whether it is network or service. If the connection is refused, I check `systemctl status nginx` and `ss -tlnp`. If I get a 502, I check the upstream application and the Nginx `error.log`. I also check disk and memory, because a full disk or an OOM kill often stops services. Then I look at what changed recently, apply the smallest safe fix, and confirm with `curl` and the logs.

Scenario 2 **"Disk usage hit 100%. What do you do?"**

Model answer: I confirm which filesystem is full with `df -h` and check inodes with `df -i`. I find the large directories with `du`, and check `lsof +L1` for deleted files still held open. I free space safely: compress or rotate logs, or restart the process holding a deleted file. I do not delete files I do not understand. Then I fix the cause, such as missing log rotation, and add an alert below 80%. On AWS, I can grow the EBS volume online if more space is genuinely needed.

12 Top Interview Traps

Short questions with a "wrong first answer" that interviewers use to test real experience.

Q1. `free` **shows almost no free memory. Is the server out of memory?**

Usually not. Linux uses spare RAM as page cache. Look at the **available** column, swap activity in `vmstat` (`si` / `so`), and OOM messages in `journalctl -k`.

Q2. **Load average is high, but the CPU is mostly idle. Why?**

Load also counts D-state tasks waiting on I/O. Check the `wa` and `b` columns in `vmstat` and run `iostat -xz`. The usual causes are slow disks, an EBS volume at its IOPS limit, or a hung NFS/EFS mount.

Q3. `df` says 100% but `du` finds much less. Why?

A process still has deleted files open (`lsof +L1`), or files are hidden under a mount point. Restart or signal the process; deleting more files will not help.

Q4. "No space left on device", but `df -h` shows free space?

The filesystem has run out of inodes (`df -i`), usually because of millions of small files such as sessions, cache or mail queues.

Q5. A deployment gets "permission denied". Is `chmod 777` a fix?

No. It gives every user, and every compromised process, write access. Find the blocking path with `namei -l` , then fix ownership or group permissions, and check SELinux labels on RHEL.

Q6. Restart or reload after a config change?

Validate first (`nginx -t`), then reload. Reload keeps existing connections, and a broken config leaves the old one running. A restart with a broken config takes the service down.

Q7. `dig` returns the right IP, but the application connects to the wrong one. Why?

Applications use the system resolver, which checks `/etc/hosts` first according to `/etc/nsswitch.conf` . `dig` queries DNS directly. Compare with `getent hosts` .

Q8. Why can't you see the EC2 public IP in `ip addr` ?

The internet gateway maps (NATs) the public IP to the private IP. Only the private IP is configured on the interface. Get the public IP from instance metadata.

Q9. You raised `nofile` in `limits.conf` , but the service still hits "Too many open files". Why?

`limits.conf` is applied by PAM at login. systemd services ignore it. Set `LimitNOFILE=` in a unit drop-in, restart the service, and verify in `/proc/PID/limits` .

Q10. A script works manually but fails from cron. Why?

Cron runs with a minimal `PATH` , no profile files and a different working directory, and `%` in a crontab line means newline. Use absolute paths, set `PATH` , escape `\%` , and redirect output with `2>&1` to see errors.

Q11. The pipeline is green, but the site is down. How?

The pipeline only checks the exit codes of the commands it ran. Deployments must end with a real health check that fails the job and triggers a rollback.

Q12. The service is "active (running)", but users get errors. Is it healthy?

No. "Running" only means the process exists. Check the application itself: `curl` the health endpoint, read the application logs, and check its dependencies (database, DNS, disk).

13 Final Checklist and Takeaways

Area	Can you, without notes...	Ready
Architecture	Explain kernel vs user space, system calls and the boot sequence?	☐
Permissions	Explain modes, umask, special bits and ACLs, and fix "permission denied" safely?	☐
Processes	Explain states, signals, zombies and D state?	☐
systemd	Write a unit file, read `status` and `journalctl`, explain reload vs restart?	☐
Storage	Partition, format, mount, add to fstab and grow a filesystem (including LVM)?	☐
Networking	Debug DNS, ports, routes and firewalls, and explain refused vs timed out?	☐
Logs and monitoring	Find the right log fast and summarise it with `awk` / `sort` / `uniq` ?	☐
Bash	Write a script with arguments, exit codes and real error handling?	☐
Containers	Explain namespaces, cgroups, OOMKilled and CrashLoopBackOff?	☐
AWS	Explain EC2 networking, EBS, IAM roles, IMDSv2 and Session Manager?	☐
Troubleshooting	Walk through a production incident using evidence, without blindly restarting services?	☐

Takeaways

- ✓ Lead with the short answer, then add the "why" and one real example.
- ✓ Name the command, what you expect to see, and what the result proves.
- ✓ Know the classic traps: available vs free memory, I/O load, deleted-but-open files, inodes, `/etc/hosts`, PAM limits vs systemd.
- ✓ Always mention safety: validate before reload, back up before editing, never `chmod 777`, never `kill -9` first.
- ✓ Cover both distribution families: apt/dnf, ufw/firewalld, AppArmor/SELinux, ext4/XFS.
- ✓ On AWS, check the cloud layer (security groups, NACLs, roles, AZs) as well as Linux.

PRACTICAL EXERCISES

1. Pick any five rows from this cheatsheet at random and answer each aloud in under 60 seconds.
2. For each trap in Section 12, reproduce it in your lab (most are in Labs 4–9) and capture the output.
3. Answer Scenario 1 and Scenario 2 using the six-step framework, then compare with the model answers.
4. Explain one Ubuntu vs RHEL difference and one AWS angle for every section.
5. Ask a friend to break your lab server, then talk through the investigation as if in an interview.

INTERVIEW TIP

Don't just recite commands. Explain what you check first, why you run each command, how you read its output, and what you would do next. Interviewers are testing your method, not your memory.

WHAT'S NEXT

Practise it: Linux Lab Exercises Workbook

Revision builds recall, but practice builds confidence. Rerun the Capstone troubleshooting scenarios and explain each one aloud.

END OF CHEATSHEET